ROS 2

Lania Laura Quadros
School of Mechanical
Manipal Institue of Technology
Manipal, India
quadroslania2@gmail.com

***Abstract*—This is an overview of the concepts, tools, communication architecture and development workflow of ROS 2. The concepts of nodes, topics, services, actions, parameters, custom interfaces, workspaces, packages, and the colcon build system are discussed along with the relevant ROS 2 commands used to inspect and manage these components. The report also includes Quality of Service (QoS) policies and their role between publishers and subscribers. To demonstrate the practical application of these concepts, two ROS 2 questions were implemented and explained in this report along with their rqt graphs. The first question uses a custom message to transmit angular velocity and wheel radius between a publisher and subscriber, allowing the subscriber to calculate the longitudinal wheel speed. The second question implements a three node communication system that determines whether the user inputted a palindrome string and communicates the result between the nodes.**

Keywords—ROS 2, Nodes, Topics, Services, Actions, Parameters, Custom Interfaces, Colcon, Workspace, Package, QoS Settings, ROS 2 Computational Graph.

I. Command Line Interface (CLI) Tools

ROS (Robot Operating System) is an open-source ecosystem that provides the framework, tools, and libraries for building, deploying, running, and maintaining robotic applications [1].

A ROS 2 system is divided into multiple nodes, where each node performs a specific task. These nodes communicate with each other using topics, services, and actions.

A. Nodes

Each node in ROS should be responsible for a single, modular purpose. A full robotic system is comprised of many nodes working in concert. In ROS 2, a single executable (C++ program, Python program, etc.) can contain one or more nodes [1]. e.g. A camera node captures images, while a perception node processes those images. A node can contain publishers/subscribers. The publisher/subscriber are communication entities created by a node.

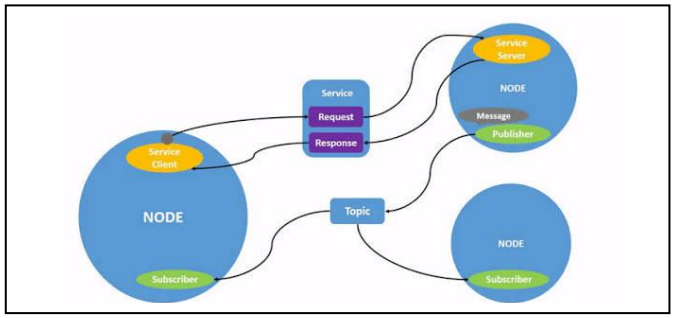


Fig. 1. ROS 2 computational graph example for nodes.

*1) Publisher:*A publisher is a node that sends (publishes) messages to a topic. It provides data that other nodes can receive. e.g. A camera node publishes images to /camera/image.

*2) Subrcirber:*A subscriber is a node that receives messages from a topic. It listens to data published by another node. e.g. A perception node subscribes to /camera/image to receive camera images.

The command ‘ros 2 run’ launches an executable form package, to run a node execute ‘ros 2 run <package_name> <executable_name>’

To see the names of the active nodes, run ‘ros 2 node list’

With the names of your nodes, you can access more information about them by running ‘ros 2 node info <node_name>

B. Topics

Topics are a named channel over which nodes exchange messages between different parts of the system.

- A publisher sends messages to a topic.
- A subscriber receives messages from a topic.

A Topic can have multiple publishers and subscribers and are generally used for a continuous data stream, such as data, images or robot velocity commands.

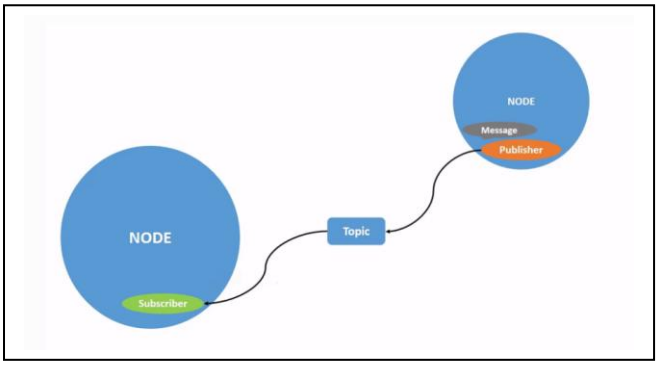


Fig. 2. ROS 2 computational graph example for topicss.

If you want to see all the active topics and their message types running, run ‘ros 2 topic list -t’

To check if they are one-to-many, many-to-one, or many-to-many, run ‘ros 2 topic info <topic_name>’

If you want to see the exact blueprint fields of any specific message type, run ‘ros 2 interface show <message_type_name>’

If you want to Echo(print) the live data streaming over a topic in real-time, run ‘ros 2 topic echo <topic_name>’

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

To view the rate at which data is published run ‘ros 2 topic hz <topic_name>’ and for its bandwidth run ‘ros 2 topic bw <topic_name>’

And finally, to list a list of topic available topics of a given type run ‘ros 2 topic find <message_type>’

C. Services

Services are another method of communication for nodes in the ROS graph. Services are based on a call-and-response model versus the publisher-subscriber model of topics [1].

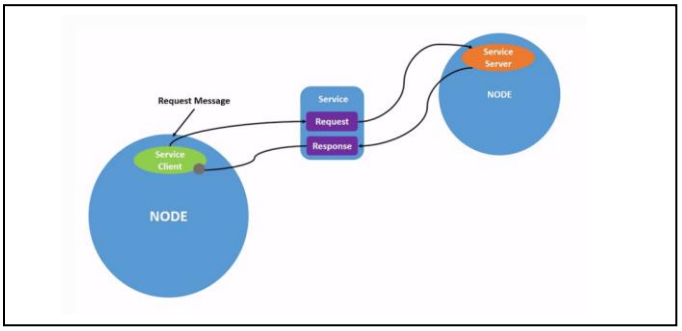


Fig. 3. ROS 2 computational graph example for services.

If you want to see all the active service types running on your system run ‘ros 2 service list -t’

If you see a service type you don’t recognize and want to see its exact blueprint run ‘ros 2 interface show <type_name>’

TABLE I. DIFFERNCE BETWEEN A TOPIC AND A SERVICE

Feature	***Topic***	***Service***
Pattern	Publish/ Subscribe (One-way)	Request/ Response (Two-way)
Communication Style	Asynchronous (Publish and forget; doesn’t wait for anything)	Synchronous/ Blocking (Client waits until server answers)
Relationship	Many-to-Many (Multiple nodes can publish or subscribe)	Many-to-One (Many clients can call, but only one server responds)
File Extension	Defined using a .msg file	Defined using a .srv file
Best Used For	Continuous, real-time streaming data	Quick, occasional tasks or computations
Example	LiDAR, Camara feed, Motor speeds	Toggling a camara, checking a battery level, calculations

D. Actions

Because Services are synchronous (blocking), they should never be used for continuous streams of data or long-running tasks. Doing so can freeze your node's execution loop and cause the robot to lag or become unresponsive. For long-running tasks that can be cancelled, an Action interface should be used instead because actions allow you to execute long running tasks, provide regular feedback, and are cancelable.

A robot system would likely use actions for navigation. An action goal could tell a robot to travel to a position. While the robot navigates to the position, it can send updates along the way (i.e. feedback), and then a final result message once it’s reached its destination.

If you want to identify all the actions in the ROS graph run ‘ros 2 action list -t’

If you want the structure of the goal, structure of the result and the structure of the feedback run ‘ros 2 interface show <action_type_name>’

To send an action goal run ‘ros 2 action send_goal <action_name> <action_type> <values>’ and if you want ot see feedback add ‘--feedback’ in the end.

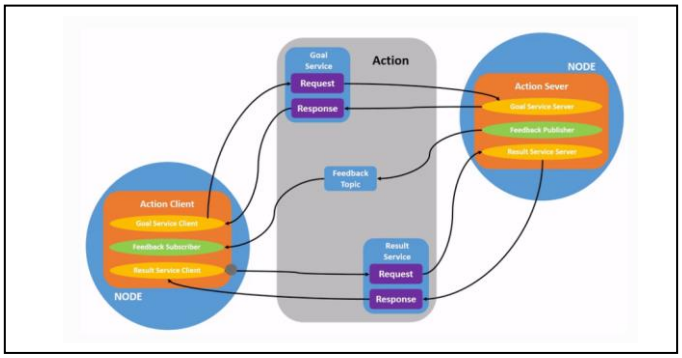


Fig. 4. ROS 2 computational graph example for actions.

E. Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, Booleans, strings, and lists. In ROS 2, each node maintains its own parameters [1].

To see the parameters belonging to your nodes, run ‘ros 2 param list’

To determine a parameter’s type run ‘ros 2 param get <node_name> <parameter_name>’

To change a parameters value run ‘ros 2 param set <node_name> <parameter_name> <value>’

Here are some examples using ‘ros 2 param set’:

1) Screen Settings: Changing ‘background_r’, ‘background_g’ or ‘background_b’ to swap the canvas color in turtlesim.

2) Speed limits: Setting a ‘max_speed’ parameter to ‘2.0’ so your robot doesn’t crash during testing.

3) Safty Switches: Toggling an ‘enable_safty_stop’ parameter to ‘true’ or ‘false’.

You can view all of the node’s current parameter values by running ‘ros 2 param dump <node_name>’ and if you want to save your current configuration into a file then run ‘ros 2 param dump <node_name> > <parameter_file>’

You can load the parameter forma file to a currently running node by running ‘ros 2 param load <node_name> <parameter_file>’

II. CLIENT LIBRARIES

A. Using Colcon to build pacakages

Colcon is the build tool commonly used in ROS 2 to build packages within a workspace. It identifies packages in the workspace, determines their dependencies, builds them in the

appropriate order, and generates the required build and installation artifacts.

A ROS 2 workspace can contain multiple packages, allowing several related components to be developed and built together. The command colcon build is executed from the root of the workspace and builds the packages present in the src directory.

After a successful build, the generated environment should be sourced so that ROS 2 can locate the newly built packages and executables.

To build run the following three commands:

- $ cd ~/ros 2_ws
- $ colcon build
- $ source install/setup.bash

To build only a selected package and its required dependencies run 'colcon build --package-select <package_name>'

B. Creating a Workspace

A ROS 2 workspace is a directory used to organize and develop ROS 2 packages. The workspace provides a common location in which packages can be created, built, and sourced. A typical workspace contains a src directory for source packages and directories generated during the build process.

A workspace can be created using the following commands:

- $ mkdir -p ~/ros 2_ws/src
- $ cd ~/ros 2_ws

After building the workspace using colcon, additional directories aree generated:

1) scr/: Contains the source code and ROS 2 packages.
2) build/: Contains intermediate build files genrated by colcon.
3) install/: Contains installed packages, executables and environment setup files.
4) log/: Contains loges generated during the build process.

C. Creating a Package

A ROS 2 package is the basic unit used to organize ROS 2 software. A package can contain nodes, source code, custom interfaces, launch files, configuration files and other resources required by a ROS 2 application.

Packages are created inside the src directory of a workspace.

For creating a package run the following commands:

- $ cd ~/ros 2_ws
- $ ros 2 pkg create --build-type ament_python <package_name> or ros 2 pkg create --build-type ament_cmake <package_name>
- $ cd ~/ros 2_ws
- $ colcon build
- $ source install/setup.bash

To verify the package using run 'ros 2 pkg list' by default it will show the following:

1) Package.xml: contains package metadata (package name, version, description, maintainer, licence) and dependencies.
2) Setup_py: used to configure installion of python ROS 2 package and its executable enteery points.
3) <package_name>: This is the python package director. It contains the actual python source code for your ROS 2 nodes. Under this come more files namely:
a) __init__.py: this tells Python that the directory chould be tread as a pyhton package.
b) And you other node implementations eg. Publisher.py and subscriber.py
4) resource/: It purpose is to allow ROS 2's package discovery and python packaging system to identify thre package correctly after installation.
5) setup.cfg: This provides additional configuration for the python package. It halps ensure that installed python executables are placed where ROS 2 expects them.

D. Implementing custom interfaces

ROS 2 interfaces define the structure of data exchanged between nodes. ROS 2 supports three types of interfaces:

1) A message interface (.msg): defines the data transmitted through a topic.
2) A service interface (.srv): defines a request and response.
3) An action interface (.action): defines a goal, feedback and result.

III. Quality of Service (QoS) Settings

QoS in ROS 2 defines policies that control how messages are delivered between publishers and subscribers. QoS allows communication to be configured according to the requirements of a robotic application. Different sensors and control systems may require different levels of reliability, message history and delivery behaviour.

One important aspect of QoS is that two profiles, e.g., a publisher's profile and a subscription's profile, may be incompatible, meaning they cannot communicate. It is up to the implementation to decide if two QoS profiles are compatible [1].

The major QoS policies and their settings are:

1) History:
a) Keep last: only stores up to N samples, configures via the queue depth options [2].
b) Keep all: stores all samples, subject to the configured resource limits of the underlying middleware [2].
2) Depth:
a) Queue size: only honored if the "history" policy was set to "keep last" [2].
3) Reliability:

a) Best effort: attempt to deliver samples, but may lose them if the network is not robust [2].

b) Reliable: guarantee to deliver samples, ut may lose them if the network is not robust [2].

TABLE II. COMPATIBILITY OF RELIABLE QOS POLICIES

Publisher	***Subscription***	***Compatible***
Best effort	Best effort	Yes
Best effort	Reliable	No
Reliable	Best effort	Yes
Reliable	Reliable	Yes

4) Durability:

a) Transient local: the publisher becomes responsible for persisting samples for "late-joining" subscriptions [2].

b) Volatile: no attempt is made to persist samples [2].

TABLE III. COMPATIBILITY OF DURABILITY QOS POLICIES

Publisher	***Subscription***	***Compatible***	***Result***
Volatile	Volatile	Yes	New messages only
Volatile	Transient local	No	No communication
Transient local	Volatile	Yes	New messages only
Transient local	Transient local	Yes	New and old messages

5) Deadline:

a) Duration: the expected maximum amount of time between subsequent messages being published to a topic [2].

TABLE IV. COMPATIBILITY OF DEADLINE QOS POLICIES

Publisher	***Subscription***	***Compatible***
Default	Default	Yes
Default	X	No
X	Default	Yes
X	X	Yes
X	Y (where y > x)	Yes
X	Y (where y < x)	No

[a.] Assume x and y are arbitrary valid duration values

6) Lifespan:

a) Durations: the maximum amount of time between the publishing and the reception of a message without the message being considered stale or expired(silently dropped messages that are effectively never dropped) [2].

7) Liveliness:

a) Automatic: the system will consider all of the node publishers to be alive for another "lease duration" when any one of its publishers has published a message [2].

b) Manual by topic: the system will consider the publisher to alive for another "lease duration" if it manually asserts that it is still alive [2].

TABLE V. COMPATIBILITY OFLIVELINESS QOS POLICIES

Publisher	***Subscription***	***Compatible***
Automatic	Automatic	Yes
Automatic	Manual by topic	No
Manual by topic	Automatic	Yes
Manual by topic	Manual by topic	Yes

8) Lease duration:

a) Duration: the maximum period of time a publisher has to indicate that it is alive before the system considers it to have lost liveliness (indication of failure) [2].

TABLE VI. COMPATIBILITY OFLEASE DURATION QOS POLICIES

Publisher	***Subscription***	***Compatible***
Default	Default	Yes
Default	X	No
X	Default	Yes
X	X	Yes
X	Y (where y > x)	Yes
X	Y (where y < x)	No

[a.] Assume x and y are arbitrary valid duration values

For each of the policies that is not a duration, there is also the option of "system default", which uses the default of the underlying middleware. For each of the policies that is a duration, there also exists a "default" option that means the duration is unspecified, which the underlying middleware will usually interpret as an infinitely long duration [2].

IV. TASK SOLUTIONS

A. Question 1

Create a message package by running 'ros 2 pkg create –build-type ament_cmake task_one_interfaces' then create a file 'msg' in it with a custom ROS 2 message named TaskOne.msg containing two floating-point values, ang_vel and radius then build it.

Create a node package by running 'ros 2 pkg create –build-type ament_python task_one_nodes –dependencies rclpy task_one_interfaces' with a publisher node that obtains the angular velocity (ω) and wheel radius (r) from the user through the terminal and publishes the values on the /taskone topic and a subscriber node that receives the message and calculates the longitudinal velocity (V) of the wheel using the relationship given in (1).

$$V = \omega r \tag{1}$$

Open 3 terminals:

1) First terminal: run the publisher node and enter a angular velocity and radius of the wheel.

2) Second terminal: run the subscriber node and it will receive the angular velocity and radius of the wheel from the

publisher node and then publish the longitudunal velocity of the wheel.

3) Third terminal: open the rqt gragh with command ‘ros 2 run rqt_graph rqt_graph’

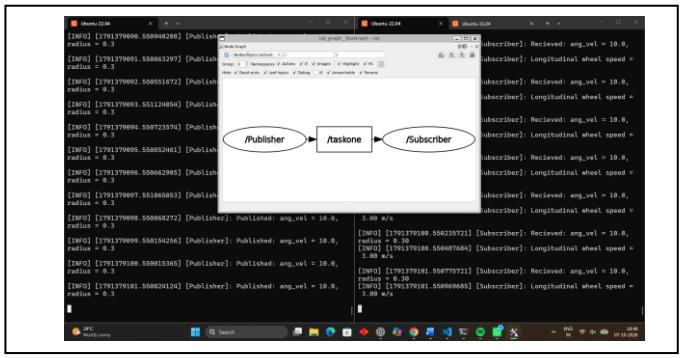


Fig. 5. Rqt graph for the first question.

B. Question 2

Create a message package by running ‘ros 2 pkg create –build-type ament_cmake task_two_interfaces’ then create a file ‘msg’ in it with a custom ROS 2 message named RT2String.msg containing one string value called data.

Node 1 accepts a string from the user and publishes it using the custom RT2String message. Node 2 subscribes to the string and determines whether it is a palindrome. It publishes 1 if the string is a palindrome and 0 if it is not a palindrome. Node 3 receives this result from node 2 and converts it into the corresponding "Yes" or "No" response. The response is then published back to Node 2.

Open 4 terminals:

1) First terminal: run node 1 and enter a string.

2) Second terminal: run node 2 and it will output 1/0 depending if the string publisher by node1 is palindrome or not repectively. And once node3 starts running it will also simultenuosly publish what is recived by node3.

3) Third terminal: run node 3 which will recived 1/0 from node 2 and publish Yes/No respectively.

4) Fourth terminal: open the rqt gragh with command ‘ros 2 run rqt_graph rqt_graph’

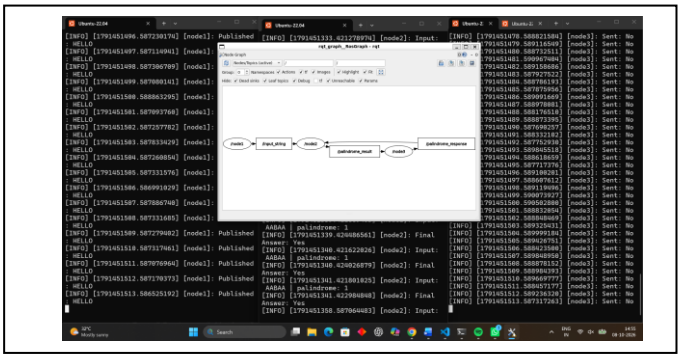


Fig. 6. Rqt graph for the second question.

ACKNOWLEDGMENTS

I extend my deepest gratitude to the seniors in the Driverless domain of the student project, Formula Manipal, whose resource and assistance have been invaluable, I would like also like to acknowledge my peers for their camaraderie and the stimulating discussions that inspired me throughout my task 2 journey

REFERENCES

[1] ROS 2 Doucumentation : humble. [Online]. Available: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html

[2] ROS 2 Documentation : Humble. [Online]. Available: https://ros2docs.robook.org/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html